0. **Code:** uppalh8095 Uppal Harsh.zip

1. MidTerm/A. Larger Running Averages/A. Larger Running Averages/Program.cs
2. MidTerm/B.Nested Loop/B.Nested Loop/Program.cs

1. MidTerm/A. Larger Running Averages/A. Larger Running Averages/Program.cs ▶

```
using System.Runtime.ConstrainedExecution;

namespace A._Larger_Running_Averages
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] array = {
                7,
                3,
                1,
                8,
                2,
                4,
                5,
                0,
                9
            };
            Console.WriteLine(LargerRunningAverages(array));
        }
        public static decimal LargerRunningAverages(int[] array)
        {
            if (array.Length == 0 || array == null)
            {
                return 0;
            }
            int sum = 0;
            int final_sum = 0;
            int final_count = 0;
            decimal running_average = 0;
```

swapped.

A3

1. MidTerm/A. Larger Running Averages/A. Larger Running Averages/Program.cs

```
            decimal final_average = 0;

            for (int i = 1; i < array.Length; i++)
            {
                //calculate previous sum
                sum += array[i - 1];
                // calculate previous average and compare it to the current one
                running_average = (decimal) sum / i;

                //add the number grater than previous average to the final sum and increment the count value
                if (array[i] > running_average)
                {
                    final_sum += array[i];
                    final_count++;
                }
            }

            if (final_count > 0)
            {
                final_average = (decimal) final_sum / final_count;
                return final_average;
            }
            else return 0;
        }
    }
}
```

A2b

A2c

A2

A3

2. MidTerm/B.Nested Loop/B.Nested Loop/Program.cs

```
using System;
using System.Reflection;

namespace B._Nested_Loop
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] array = {
                7,
                3,
                1,
                8,
                2,
                4,
                5,
                0,
                9
            };
            int[] array2 = {
                1,
                2,
                2,
                1,
                3
            };
            int[] result = NestedLoop(array2);
            Console.WriteLine(String.Join(", ", result));
        }
        public static int[] NestedLoop(int[] array)
```

131

2. MidTerm/B.Nested Loop/B.Nested Loop/Program.cs

```
        {
            if (array == null)
            {
                return null;
            }
            if (array.Length == 0)
            {
                return null;
            }

            int new_array_length = (array.Length * (array.Length + 1)) /
              2;
            int[] result = new int[new_array_length];
            int min = int.MaxValue;
            int count = 0;
            int start = 0;
            int end = 0;
            int temp_start = 0;
            for (int i = 0; i < new_array_length; i++)
            {
                if (count < array.Length)
                {
                    result[count] = array[count];
                    if (array[count] < min)
                    {
                        min = array[count];
                    }
                    count++;
                    end = count;
                }
```

need to be initialized in the loop.

copy first.

13

Finding smallest.

ok.

2. MidTerm/B.Nested Loop/B.Nested Loop/Program.cs

```
            else if (count < new_array_length)
            {
                start = temp_start;
                for (int index = start + 1; index < end; index++)
                {
                    if (result[index] < result[index])
                    {
                        min = result[index];
                    }
                }
                int c = 0;
                temp_start = count;
                for (int j = start; j < end; j++)
                {
                    if (count == new_array_length)
                    {
                        break;
                    }

                    if (result[j] == min && c == 0)
                    {
                        c++;
                    }
                    else if (result[j] < min && c > 0)
                    {
                        result[count] = result[j];
                        count++;

                    }
                    else if (result[j] > min)
```

when previous block exists

int minIndex=-1;

previous block start

B4

minIndex= index

you could use this instead of var c.

Find smallest

current block start.

end refers to the last Index +1 of the previous block.

if everything works this is unnecessary

B4

You are either doing these two

B3

So if(——) else { Copy } is enough.

2. **MidTerm/B.Nested Loop/B.Nested Loop/Program.cs**

```
                {
                    result[count] = result[j];
                    count++;
                }
            }
            end = count;

        }
    }
    return result;

}
}
}
```

B3

repeated code

previous block current block

each time

min = max value

Good work.

I think your code will work if your min is re-initialized each time when you check the min in the previous block. otherwise, you will not get the min for the 2nd, 3rd, etc blocks

B5